Gradient Descent

Gradient Descent is an optimization algorithm to find local maxima/minima. It can be used to update the weight vector so as to minimize the error. Gradient of the input vector is the partial derivative of vector w.r.t each input component.

$$\nabla\phi\,(x, y, z) = \frac{\partial\phi}{\partial x}\,\hat{\mathbf{x}} + \frac{\partial\phi}{\partial y}\,\hat{\mathbf{y}} + \frac{\partial\phi}{\partial z}\,\hat{\mathbf{z}}$$

The gradient specifies the direction that produces the steepest increase (gradient ascent) in input vector. The negative of gradient gives the direction of steepest decrease (gradient descent).

The gradient descent algorithm is as follows:

GRADIENT-DESCENT(*training_examples*, η)

Each training example is a pair of the form $(\vec{x}, t)$*, where* $\vec{x}$ *is the vector of input values, and t is the target output value.* η *is the learning rate.*

- Initialize each w_i to some small random value
- Until the termination condition is met, do
    - Initialize each Δw_i to zero
    - For each $(\vec{x}, t)$ in *training_examples*, do
        - Input the instance $\vec{x}$ to the unit and compute the output *o*
        - For each linear unit weight w_i, do

$$\Delta w_i \leftarrow \Delta w_i + \eta(t - o)x_i$$

    - For each linear unit weight w_i, do

$$w_i \leftarrow w_i + \Delta w_i$$

The gradient descent algorithm given above has some limitations:

i. Converging to a local minimum can sometimes be quite slow.
ii. If there are multiple local minima, then there is no guarantee that the procedure will find the global minimum.

To handle these limitations, there is a variant of gradient descent called as incremental/stochastic gradient descent.

STOCHASTIC-GRADIENT-DESCENT(*training_examples*, η)

Each training example is a pair of the form $(\vec{x}, t)$*, where* $\vec{x}$ *is the vector of input values, and t is the target output value.* η *is the learning rate.*

- Initialize each w_i to some small random value
- Until the termination condition is met, do
    - Initialize each Δw_i to zero
    - For each $(\vec{x}, t)$ in *training_examples*, do
        - Input the instance $\vec{x}$ to the unit and compute the output *o*
        - For each linear unit weight w_i, do

$$w_i \leftarrow w_i + \eta(t - o)x_i$$

Regression (Ordinary Least Squares Regression)

Regression means approximating a real-valued target function.

Let the training dataset be:

X1	X2	Y
2.7	2.5	0
1.4	2.3	0
7.6	2.7	1
5.3	2.0	1

We have to train the model to predict the value of Y based on values of X1 and X2.

For this dataset, the linear regression has three coefficients, for example:

$$\textbf{output} = b_0 + b_1 * x_1 + b_2 * x_2$$

The job of the learning algorithm will be to discover the best values for the coefficients (b_0, b_1 and b_2) based on the training data, so as to minimize the error/loss.

The error/loss function we will use is

$$E = (1/2m) * \Sigma (Z_i - Y_i)^2$$ (m is the number of training examples)

The function given above is called Ordinary Least Squares.

This function can be minimized using gradient descent.

So, we will try to find out the gradient of OLS.

$$\frac{\partial E}{\partial w_c} = \frac{\partial}{\partial w_c} \frac{1}{2} \sum_{i \in D} (t_i - p_i)^2$$

$$\frac{\partial E}{\partial w_c} = \frac{1}{2} \frac{\partial}{\partial w_c} \sum_{i \in D} (t_i - p_i)^2$$

$$\frac{\partial E}{\partial w_c} = \frac{1}{2} \sum_{i \in D} 2(t_i - p_i) \frac{\partial}{\partial w_c} (t_i - p_i)$$

$$\frac{\partial E}{\partial w_c} = \sum_{i \in D} (t_i - p_i) \frac{\partial}{\partial w_c} (t_i - p_i)$$

$$\frac{\partial E}{\partial w_c} = \sum_{i \in D} (t_i - p_i) \frac{\partial}{\partial w_c} (t_i - w x_i)$$

$$\frac{\partial E}{\partial w_c} = \sum_{i \in D} (t_i - p_i)(-x_{ci})$$

$$\frac{\partial E}{\partial w_c} = -\sum_{i \in D} (t_i - p_i)(x_{ci})$$

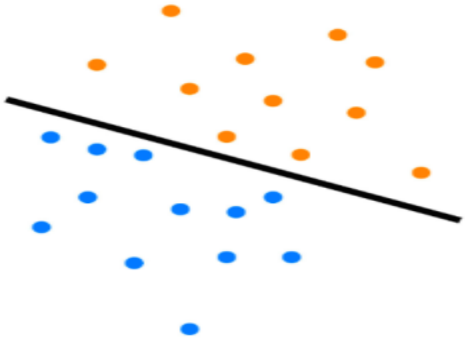
So ,the gradient descent rule becomes

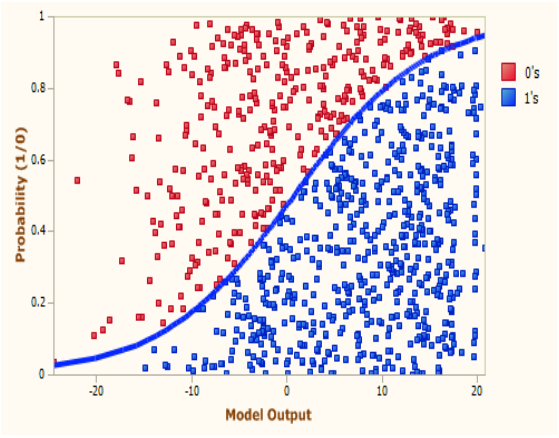
$$\Delta w_c = -\alpha \frac{\partial E}{\partial w_c} = \alpha \sum_{i \in D} (t_i - p_i) x_{ci}$$

This rule is called as "**delta rule**" i.e. delta rule is a gradient descent rule for updating the weights of inputs to minimize ordinary least squares error.

Discriminative Classifiers

A discriminative classifier trains a function called as discriminative function to classify the inputs. This discriminative function is a hyper-plane (subspace of one dimension less than the ambient space) which divides the space into sub-parts. The hyper-plane can be linear or non-linear. For example, if the ambient space is a 2D space then the linear discriminant is a straight line, whereas a non-linear discriminant is a curve. Examples of discriminative classifiers are logistic regression, perceptron, and support vector machine (SVM).





Linear Discriminant: $f = w_0+w_1x_1+\ldots+w_nx_n = b+w^Tx$

The discriminant assigns a coefficient called as "weight" to each input variable. The set of these coefficients is called as "weight vector". There is an extra coefficient w_0 which is not associated with any input, this is called as "bias" which can be set to handle cases where a non-zero output is required when all inputs are zeroes. The weight w_0 is also denotes by "*b*" which stands for bias.

During the training process the weights of the discriminant are adjusted until the error between predicted values and target values for all the training instances is minimized. Once the discriminant is trained, it performs the maximum likelihood estimate of the class to which the input may belong.

Discriminative classifiers are *binary* i.e. they can handle only two class problems. However they can be used to solve multi-class problem by decomposing the problem into multiple binary classification problems. The discriminative classifiers like single-layer perceptron and support vector machines are *linear*, but SVM can handle non-linear data by "kernel trick" which transforms the feature space into a binary classification problem. Multi-layer perceptron (Artificial Neural Networks) are able to handle to handle non-linear and multi-class problems.

1. Logistic Regression (Binary Linear Classification)

Logistic Regression is a probabilistic classifier as it learns the conditional probability distribution P(C|X) from the data during the training step (probability of a class given the data instance), therefore it is also called as probabilistic conditional model.

For a two-class problem, the decision boundary lies where the prediction probability is 0.5.

Therefore, the response function used by logistic regression is:

$$f(wx) = \frac{1}{1+e^{-(w^T x+b)}}$$

f(wx)=0.5 when $w^T x+b=0$

i.e. decision boundary is obtained when $w^T x+b=0$, hence the boundary is a hyper-plane. Just like linear regression, weights must be found that fit the training data well.

If we have two classes +1 and -1, the probabilities would be P and 1-P.

The logistic function satisfies the property:

$$f(-z) = 1 - f(z)$$

Therefore, if

$$P(t=1 \mid x; w) = \frac{1}{1+e^{-(w^T x+b)}}$$

then

$$P(t=-1 \mid x; w) = 1 - P(t=1 \mid x; w) = \frac{1}{1+e^{(w^T x+b)}}$$

So, we can write

$$P(t \mid x; w) = \frac{1}{1+e^{-t(w^T x+b)}}$$

The likelihood function will then be

$$l(w,b) = \prod_{i=1}^{n} \frac{1}{1+e^{-t(w^T x+b)}}$$

The log-likelihood function will be

$$LL(w,b) = \sum_{i=1}^{n} \log\left(1+e^{-t(w^T x+b)}\right)$$

So, we need to minimize

$$L(w,b) = -\sum_{i=1}^{n} \log\left(1+e^{-t(w^T x+b)}\right)$$

This can be done by gradient descent. So, we need to find the gradient of the loss function.

$$\frac{\partial}{\partial w}(\log[1+e^{-yw^T x}]) = \frac{1}{1+e^{-yw^T x}} \frac{\partial}{\partial w}[1+e^{-yw^T x}] = \frac{1}{1+e^{-yw^T x}} \frac{\partial}{\partial w}[e^{-yw^T x}]$$

$$= \frac{e^{-yw^T x}}{1+e^{-yw^T x}} \frac{\partial}{\partial w}(-ywx) = \frac{e^{-yw^T x}}{1+e^{-yw^T x}}(-yx)$$

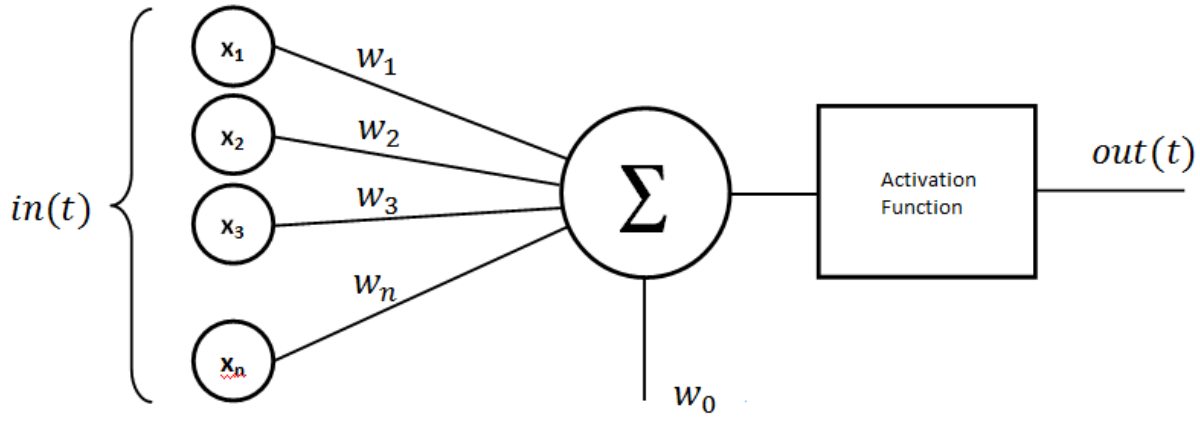
$$= -y\left(\frac{1+e^{-yw^T x}-1}{1+e^{-yw^T x}}\right)x = -y\left(1-\frac{1}{1+e^{-yw^T x}}\right)x$$

So the weight update rule for gradient descent becomes $\Delta w = \alpha\left(-\frac{\partial L}{\partial w}\right) = \alpha y\left(1-\frac{1}{1+e^{-yw^T x}}\right)x$

2. Perceptron (Binary Linear Classification)

A perceptron is the simplest model of Artificial Neural Network. It consists of a single artificial neuron with Heaviside Step function as the activation function.

The Heaviside step function is written as:
f(x) = +1 if x>θ
= -1 if x≤θ



The perceptron is a linear binary classifier. The training phase of perceptron performs multiple iterations on the training data points. Each iteration goes through all the training instances, and if a misclassified instance is encountered, the parameters of the hyperplane are changed so that the misclassified instance moves closer to the hyperplane or maybe even across the hyperplane onto the correct side.

Perceptron Learning Algorithm

Set all the weights to zero
Until all the instances in the training data are classified correctly
    For each instance I in the training data
        If I is classified incorrectly by the perceptron
            If I belongs to first class add it to the weight vector
            else subtract it from the weight vector

Let's suppose the target output for a training instance I is t_i, and the output predicted by the discriminant is p_i. As the output is either +1 or -1, so if there is misclassification then $t_i * p_i = -1$ otherwise $t_i * p_i = 1$ (1*1=1 as well as -1*-1=1). So, if there are M misclassified points then the total error would be $-M$.

Instead of taking parameter t_i*p_i, we take the parameter $-(t_i*p_i)$, so that for misclassified points $-(t_i*p_i)=+1$ and for correctly classified points $-(t_i*p_i)=-1$. So, for M misclassified points the total error would be $+M$. The aim of the training stage is to minimize this error.

A function which calculates the total error over the entire training set is called as "loss function". The loss function of perceptron can be written as:

$$L_p = \sum_{i=0}^{N} \max\left[0, -(t_i \times p_i)\right] = \sum_{i=0}^{N} \max[0, -(t_i \times (w^T x + b))]$$

The loss function denotes that for each input instance, the value is maximum of $[0, -(t_i*p_i)]$, as $-(t_i*p_i)=-1$ for correctly classified instances, so loss function takes 0 for them; however $-(t_i*p_i)=+1$ for misclassified instances, so loss function takes +1 for each misclassified instance.

The aim of perceptron learning algorithm is to minimize this loss function. This can be done by gradient descent. The gradient of this loss function comes out to be

$$\frac{\partial L_p}{\partial w} = \frac{\partial[-(t_i \times w^T x)]}{\partial w} = -t_i x_i$$

As, we want to perform gradient descent, therefore, we will use $-\frac{\partial L_p}{\partial w} = t_i x_i$

So, the weight update rule becomes

$$\Delta w = \alpha \times \sum_{d=1}^{D} t_i x_{id}$$

where D is the number of attributes in the instance (or the dimensions of the dataset).
So, if the instance belongs to first class (+1) then its attribute values are added to weight vector, otherwise its attribute values are subtracted from the weight vector.

Let's try to train a perceptron for a two-input NAND gate.

X1	**X2**	**T**
0	0	1
0	1	1
1	0	1
1	1	0

We take b=1, θ=0.5 and α=0.1

Input				Initial weights			Output					Error	Correction	Final weights		
Sensor values			Desired output				Per sensor			Sum	Network					
x_0	x_1	x_2	z	w_0	w_1	w_2	c_0	c_1	c_2	s	n	e	d	w_0	w_1	w_2
							$x_0 * w_0$	$x_1 * w_1$	$x_2 * w_2$	$c_0 + c_1 + c_2$	if $s > t$ then 1, else 0	$z - n$	$r * e$	$\Delta(x_0 * d)$	$\Delta(x_1 * d)$	$\Delta(x_2 * d)$
1	0	0	1	0.3	0.1	0.1	0.3	0	0	0.3	0	1	+0.1	0.4	0.1	0.1
1	0	1	1	0.4	0.1	0.1	0.4	0	0.1	0.5	0	1	+0.1	0.5	0.1	0.2
1	1	0	1	0.5	0.1	0.2	0.5	0.1	0	0.6	1	0	0	0.5	0.1	0.2
1	1	1	0	0.5	0.1	0.2	0.5	0.1	0.2	0.8	1	-1	-0.1	0.4	0	0.1

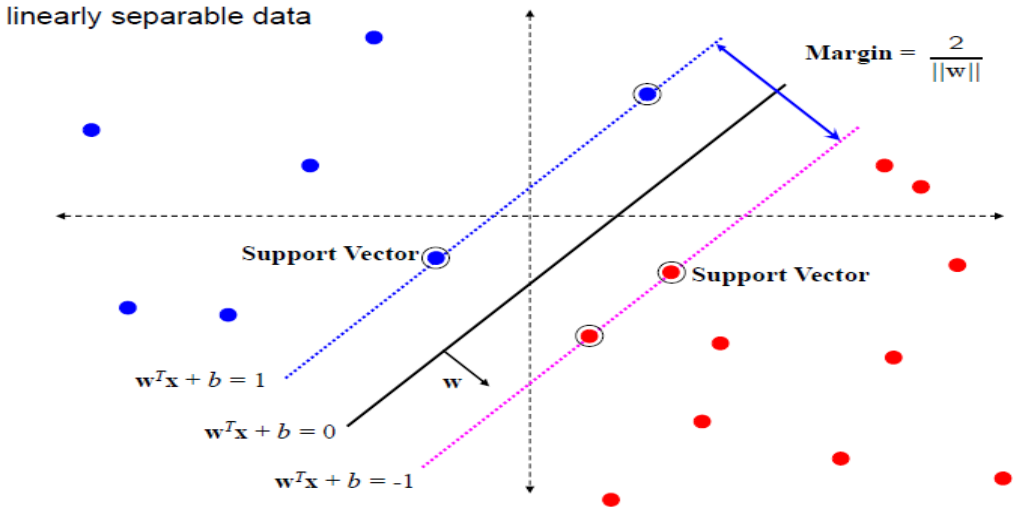
1	0	0	1	0.3	0.1	0.1	0.3	0	0	0.3	0	1	+0.1	0.4	0.1	0.1
1	0	1	1	0.4	0.1	0.1	0.4	0	0.1	0.5	0	1	+0.1	0.5	0.1	0.2
1	1	0	1	0.5	0.1	0.2	0.5	0.1	0	0.6	1	0	0	0.5	0.1	0.2
1	1	1	0	0.5	0.1	0.2	0.5	0.1	0.2	0.8	1	-1	-0.1	0.4	0	0.1
1	0	0	1	0.4	0	0.1	0.4	0	0	0.4	0	1	+0.1	0.5	0	0.1
1	0	1	1	0.5	0	0.1	0.5	0	0.1	0.6	1	0	0	0.5	0	0.1
1	1	0	1	0.5	0	0.1	0.5	0	0	0.5	0	1	+0.1	0.6	0.1	0.1
1	1	1	0	0.6	0.1	0.1	0.6	0.1	0.1	0.8	1	-1	-0.1	0.5	0	0
1	0	0	1	0.5	0	0	0.5	0	0	0.5	0	1	+0.1	0.6	0	0
1	0	1	1	0.6	0	0	0.6	0	0	0.6	1	0	0	0.6	0	0
1	1	0	1	0.6	0	0	0.6	0	0	0.6	1	0	0	0.6	0	0
1	1	1	0	0.6	0	0	0.6	0	0	0.6	1	-1	-0.1	0.5	-0.1	-0.1
1	0	0	1	0.5	-0.1	-0.1	0.5	0	0	0.5	0	1	+0.1	0.6	-0.1	-0.1
1	0	1	1	0.6	-0.1	-0.1	0.6	0	-0.1	0.5	0	1	+0.1	0.7	-0.1	0
1	1	0	1	0.7	-0.1	0	0.7	-0.1	0	0.6	1	0	0	0.7	-0.1	0
1	1	1	0	0.7	-0.1	0	0.7	-0.1	0	0.6	1	-1	-0.1	0.6	-0.2	-0.1
1	0	0	1	0.6	-0.2	-0.1	0.6	0	0	0.6	1	0	0	0.6	-0.2	-0.1
1	0	1	1	0.6	-0.2	-0.1	0.6	0	-0.1	0.5	0	1	+0.1	0.7	-0.2	0
1	1	0	1	0.7	-0.2	0	0.7	-0.2	0	0.5	0	1	+0.1	0.8	-0.1	0
1	1	1	0	0.8	-0.1	0	0.8	-0.1	0	0.7	1	-1	-0.1	0.7	-0.2	-0.1
1	0	0	1	0.7	-0.2	-0.1	0.7	0	0	0.7	1	0	0	0.7	-0.2	-0.1
1	0	1	1	0.7	-0.2	-0.1	0.7	0	-0.1	0.6	1	0	0	0.7	-0.2	-0.1
1	1	0	1	0.7	-0.2	-0.1	0.7	-0.2	0	0.5	0	1	+0.1	0.8	-0.1	-0.1
1	1	1	0	0.8	-0.1	-0.1	0.8	-0.1	-0.1	0.6	1	-1	-0.1	0.7	-0.2	-0.2
1	0	0	1	0.7	-0.2	-0.2	0.7	0	0	0.7	1	0	0	0.7	-0.2	-0.2
1	0	1	1	0.7	-0.2	-0.2	0.7	0	-0.2	0.5	0	1	+0.1	0.8	-0.2	-0.1
1	**1**	**0**	**1**	**0.8**	**-0.2**	**-0.1**	**0.8**	**-0.2**	**0**	**0.6**	**1**	**0**	**0**	**0.8**	**-0.2**	**-0.1**
1	**1**	**1**	**0**	**0.8**	**-0.2**	**-0.1**	**0.8**	**-0.2**	**-0.1**	**0.5**	**0**	**0**	**0**	**0.8**	**-0.2**	**-0.1**
1	**0**	**0**	**1**	**0.8**	**-0.2**	**-0.1**	**0.8**	**0**	**0**	**0.8**	**1**	**0**	**0**	**0.8**	**-0.2**	**-0.1**
1	**0**	**1**	**1**	**0.8**	**-0.2**	**-0.1**	**0.8**	**0**	**-0.1**	**0.7**	**1**	**0**	**0**	**0.8**	**-0.2**	**-0.1**

3. Support Vector Machine (Binary Linear Classification)

1. SVM is a discriminant-based, binary, linear classifier.
2. The discriminant is learnt by following rule:

$$w^T x_i + b \geq 1 \text{ if } y_i = +1$$
$$w^T x_i + b \leq -1 \text{ if } y_i = -1$$
i.e. $\mathbf{y_i(w^T x_i + b) \geq 1}$

3. Support Vectors are points x_i such that $\mathbf{y_i(w^T x_i + b) = 1}$
   i.e. $w^T x_i + b = +1$ for positive support vectors, and, $w^T x_i + b = -1$ for negative support vectors.



4. We have to maximize the distance between both the support vectors (to get a good generalized classifier). The vector bisecting this margin is the linear discriminant.

   As the margin b/w any two points is given by $\frac{w}{\|w\|}.(x_2 - x_1)$

   Therefore the margin b/w support vectors will be

$$\frac{w^T(x_p - x_n)}{\|w\|} = \frac{w^T x_p - w^T x_n}{\|w\|} = \frac{(1-b)-(-1-b)}{\|w\|} = \frac{2}{\|w\|}$$

5. The problem is to maximize

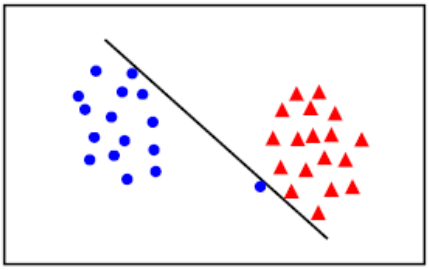
   $\frac{2}{\|w\|}$ subject to $y_i(w^T x_i + b) \geq 1$

   i.e. we have to minimize $\|w\|$

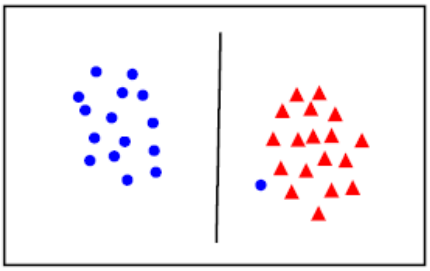
   or we have to **minimize $\|w\|^2$ subject to $y_i(w^T x_i + b) \geq 1$**

Soft-margin SVM

6. Often we need to soften our constraint to get the hyper-plane. This may be needed to handle noisy (anomalous data) otherwise it may be impossible to get a hyper-plane.



• the points can be linearly separated but there is a very narrow margin



• but possibly the large margin solution is better, even though one constraint is violated

So, we soften our constraint from $y_i(w^Tx_i+b) \geq 1$ to
$y_i(w^Tx_i+b) \geq 1-\xi_i$
where ξ is called as "slack" variable, and it is a positive constant i.e. $\xi \geq 0$.
$0<\xi\leq 1$ indicates that the point fall on the correct side of hyper-plane but b/w margin.
$\xi>1$ indicates that the point lies on the wrong-side of the hyper-plane.

The points for which $\xi>0$ are error points and this error/loss should be reduced. So, we introduce a penalty of cost $C\xi$ for any such data point. C is the regularization parameter, $C=\infty$ enforces all constraints (hard margin). C is generally small (1/N).
So, our optimization problem now reduces to

$$\min\left(\|w\|^2 + C\sum_{i=1}^{m}\xi_i\right) \textbf{ such that } y_i(x_i^Tw+b) \geq 1-\xi_i$$

We can re-write $y_i(x_i^Tw+b) \geq 1-\xi_i$ as $\xi_i \geq 1-y_i(w^Tx_i-b)$
As $\xi \geq 0$, we can re-write $\boldsymbol{\xi=max(0,1-y_it_i)}$ where $t_i=w^Tx_i-b$

So, our optimization problem becomes

$$\min\left(\|w\|^2 + C\sum_{i=1}^{N}\max(0,1-y_it_i)\right)$$

The value of C is generally taken to be 1/N.
The function $\max(0,1-y_it_i)$ is called as "hinge loss".
If $y_it_i>1$, then the point is correct (beyond margin) and doesn't contribute to the loss.
If $y_it_i=1$, then the point is correct (on margin) and doesn't contribute to the loss.
If $y_it_i<1$, then the point is a noise and contributes to the loss.

Non-Linear Discriminants

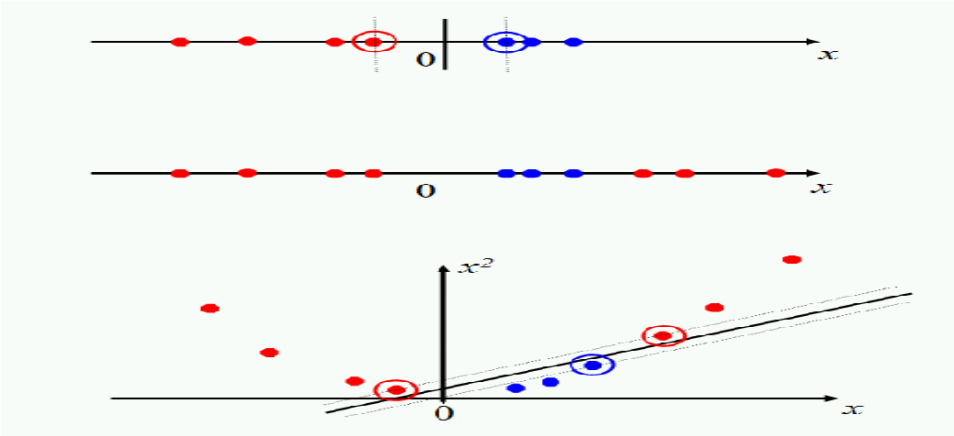


Figure 15.6: Projecting data that is not linearly separable into a higher dimensional space can make it linearly separable.

Kernel methods map the data into higher dimensional spaces in the hope that in this higher-dimensional space the data could become more easily separated or better structured.

$$K(x,y) = \langle \varphi(x), \varphi(y) \rangle$$

Using the Kernel function, the algorithm can then be carried into a higher-dimension space without explicitly mapping the input points into this space.

Consider the polynomial kernel of degree 2 defined by,
$k(x,y)=(x^Ty)^2$ where $x=(x_1,x_2), y=(y_1,y_2)$.
Thereby, the kernel function can be written as,
$k(x,y)=(x_1y_1+x_2y_2)^2=x_1^2y_1^2+2x_1x_2y_1y_2+x_2^2y_2^2$
Now, let us try to come up with a feature map Φ such that the kernel function can be written as
$k(x,y)=\Phi(x)^T\Phi(y)$.
Consider the following feature map,
$\Phi(x)=(x_1^2,\sqrt{2}x_1x_2,x_2^2)$
$\Phi(y)=(y_1^2,\sqrt{2}y_1y_2,y_2^2)$
Notice that,
$\Phi(x)^T\Phi(y)= x_1^2y_1^2+2x_1x_2y_1y_2+x_2^2y_2^2$
which is essentially our kernel function.
The new space is 3-dimensional as $\Phi(x)=(x_1^2,\sqrt{2}x_1x_2,x_2^2)$ and $\Phi(y)=(y_1^2,\sqrt{2}y_1y_2,y_2^2)$, whereas the original space was 2-dimensional as $x=(x_1,x_2), y=(y_1,y_2)$.

Radial Basis Function (RBF) kernel

A radial basis function is a real-valued function whose value depends only on the distance from the origin. Any function that satisfies the property $\phi(x)=\phi(||x||)$ is a radial function.
There are various types of RBF: Gaussian, Multi-quadratic, Inverse quadratic, etc.

Gaussian Kernel

The Gaussian kernel is an example of RBF kernel.

$$k(x,y)=\exp\left(-\frac{\|x-y\|^2}{2\sigma^2}\right)$$

The adjustable parameter *sigma* plays a major role in the performance of the kernel, and should be carefully tuned to the problem at hand. If over-estimated the exponential will behave almost linearly and the higher dimensional projection will start to lose its non-linear power. On the other hand, if under-estimated, the function will lack regularization and the decision boundary will be highly sensitive to noise in training data.

Exponential kernel

The exponential kernel is closely related to the Gaussian kernel, with only the square of the norm left out. It is also a radial basis function kernel.

$$k(x,y)=\exp\left(-\frac{\|x-y\|}{2\sigma^2}\right)$$

RBF implicitly maps every point to an infinite dimensional space.
Let us consider the Gaussian RBF kernel for points $x=(x_1,x_2), y=(y_1,y_2)$.
Then, the kernel can be written as

$$k(x,y)=\exp(-\|x-y\|^2)=\exp(-(x_1-y_1)^2-(x_2-y_2)^2)$$
$$=\exp(-x_1^2+2x_1y_1-y_1^2-x_2^2+2x_2y_2-y_2^2)$$
$$=\exp(-\|x\|^2)\exp(-\|y\|^2)\exp(2x^Ty)$$

(assuming gamma = 1). Using the taylor series you can write this as,

$$k(x,y)=\exp(-\|x\|^2)\exp(-\|y\|^2)\sum_{n=0}^{\infty}[(2x^Ty)^n/n!]$$

Now, if we were to come up with a feature map Φ just like we did for the polynomial kernel, you would realize that the feature map would map every point to an infinite vector.